

07 — KQL Cheat Sheet for AZ-500

You do not need to write production KQL for AZ-500. You need to **read a query and say what it does, pick the right table, and spot the broken line**. Twenty minutes here is enough.

1. Query shape

A query is a **table**, then a chain of operators joined by `|`. Data flows left to right and each operator receives the previous result.

```
SecurityEvent                                     // 1. tab
| where TimeGenerated > ago(24h)                  // 2. fil
| where EventID == 4625                           // 3. fai
| summarize FailedAttempts = count()             // 4. agg
    by Account, Computer, IPAddress
| where FailedAttempts > 10                       // 5. fil
| project Account, Computer, IPAddress, FailedAttempts
| sort by FailedAttempts desc                     // 7. ord
```

Reads as: *"in the last 24 hours, find accounts with more than 10 failed logons, grouped by account, computer, and source IP."*

Rule of thumb the exam likes: filter with `where` as early as possible — it is the main performance and cost optimisation.

2. Operators you must recognise

Operator	Does	Example
<code>where</code>	Filter rows	`
<code>project</code>	Select specific columns (drops the rest)	`
<code>project-away</code>	Drop named columns, keep the rest	`
<code>extend</code>	Add a calculated column	`
<code>summarize</code>	Aggregate , with <code>by</code> for grouping	`
<code>count</code>	Row count only	`
<code>distinct</code>	Unique values	`
<code>top N by</code>	Highest N	`
<code>take / limit</code>	N arbitrary rows (sampling)	`
<code>sort by / order by</code>	Sort (identical operators)	`
<code>union</code>	Combine rows from multiple tables	<code>union SecurityEvent, SigninLogs</code>
<code>join</code>	Combine columns on a key	`
<code>let</code>	Declare a variable, function, or list	<code>let window = 7d;</code>
<code>search</code>	Free-text across tables — slow, avoid	<code>search "malware"</code>
<code>parse</code>	Extract fields from a string	`

mv-expand	Expand an array into rows	`
render	Chart the result	`
evaluate	Plugins (e.g. bag_unpack)	
materialize	Cache a subquery result	

Aggregation functions inside `summarize`

`count()` · `dcount(col)` (distinct count) · `sum()` · `avg()` · `min()` / `max()` · `make_list()` / `make_set()` · `arg_max(TimeGenerated, *)` (the full row with the latest timestamp — very common) · `countif(condition)` · `percentile()`

Time functions

`ago(1h)` · `now()` · `startofday()` · `datetime()` · `todatetime()` · **`bin(TimeGenerated, 1h)`** (bucket into intervals — essential for time charts) · `between(datetime(...) .. datetime(...))`

String and comparison operators

Operator	Meaning
<code>==</code> / <code>!=</code>	Exact, case-sensitive
<code>=~</code> / <code>!~</code>	Exact, case-insensitive
<code>contains</code> / <code>!contains</code>	Substring, case-insensitive
<code>contains_cs</code>	Substring, case-sensitive (faster)
<code>has</code> / <code>!has</code>	Whole term match — faster than contains , prefer it

<code>startswith / endswith</code>	Prefix / suffix
<code>in / !in</code>	Value in a set: `
<code>matches regex</code>	Regex
<code>isempty() / isnotempty() / isnull() / isnotnull()</code>	Null and empty checks

Performance point the exam may hint at: `has > contains > search`. And filter on `TimeGenerated` first.

3. Tables you should recognise on sight

This is the highest-value part of the file — most KQL questions are really "which table?".

Table	Contains	Comes from
<code>SecurityEvent</code>	Windows security event log (logons, process creation, privilege use)	Windows Security Events connector via AMA + DCR
<code>Event</code>	Windows application/ system event logs	AMA
<code>Syslog</code>	Linux syslog	AMA
<code>CommonSecurityLog</code>	CEF logs from firewalls and appliances	CEF via AMA / log forwarder

SigninLogs	Microsoft Entra interactive sign-ins	Entra ID connector
AADNonInteractiveUserSignInLogs	Non-interactive sign-ins	Entra ID connector
AADServicePrincipalSignInLogs	Service principal sign-ins	Entra ID connector
AuditLogs	Entra directory changes (user created, role assigned, consent granted)	Entra ID connector
AzureActivity	Azure control-plane operations (who deleted the NSG)	Azure Activity connector
AzureDiagnostics	Legacy catch-all for resource diagnostic logs (Key Vault, Firewall, etc.)	Diagnostic settings
AzureNetworkAnalytics_CL	Traffic Analytics data	NSG flow logs
SecurityAlert	Alerts from Defender products and Sentinel rules	Defender connectors
SecurityIncident	Sentinel incidents	Sentinel
SecurityRecommendation	Defender for Cloud recommendations	Continuous export
ThreatIntelligenceIndicator	IOCs (domains, IPs, hashes)	TI connectors

Anomalies	Output of anomaly analytics rules	Sentinel
BehaviorAnalytics	UEBA enriched events and scores	UEBA
IdentityInfo	User profile/ enrichment data	UEBA
Heartbeat	Agent check-ins — used to find machines that stopped reporting	AMA
DeviceEvents, DeviceProcessEvents, DeviceNetworkEvents, DeviceLogonEvents	Defender for Endpoint advanced hunting	Defender XDR connector
EmailEvents, IdentityLogonEvents, CloudAppEvents	Defender for Office / Identity / Cloud Apps	Defender XDR connector
Usage	Ingestion volume per table — for cost analysis	Built in

Windows event IDs worth knowing

ID	Meaning
4624	Successful logon
4625	Failed logon (brute force)
4634 / 4647	Logoff
4672	Special privileges assigned to a new logon (admin logon)
4688	New process created

4720	User account created
4726	User account deleted
4728 / 4732 / 4756	Member added to a security group (privilege escalation)
4740	Account locked out
1102	Audit log cleared (anti-forensics)

Logon types in `SecurityEvent`: 2 = interactive, 3 = network, **10 = RemoteInteractive (RDP)**, 5 = service, 4 = batch.

4. Patterns that show up in exam-style questions

Brute force / password spray

```
SecurityEvent
| where TimeGenerated > ago(1h)
| where EventID == 4625
| summarize Failures = count(), Accounts = dcount(Account)
| where Failures > 20 or Accounts > 5
```

Many failures from one IP = **brute force**. Few failures across **many accounts** = **password spray** (that is what `dcount (Account)` is detecting).

Impossible travel / sign-ins from multiple countries

```
SigninLogs
| where TimeGenerated > ago(24h)
| where ResultType == 0 // 0 = success
```

```
| summarize Countries = make_set(LocationDetails.countryO
      CountryCount = dcount(tostring(LocationDetail
      by UserPrincipalName
| where CountryCount > 1
```

Failed Entra sign-ins by error code

```
SigninLogs
| where ResultType != 0
| summarize Count = count() by ResultType, ResultDescript
| sort by Count desc
```

ResultType == 0 is success; **50126** = invalid username or password;
 50053 = account locked; 50076/50079 = MFA required.

Who deleted or changed a resource

```
AzureActivity
| where TimeGenerated > ago(7d)
| where OperationNameValue contains "DELETE"
| where ActivityStatusValue == "Success"
| project TimeGenerated, Caller, OperationNameValue, Reso
```

AzureActivity is always the answer for "who performed this management operation".

Role assignment changes (privilege escalation)

```
AuditLogs
| where OperationName has "Add member to role"
| extend Target = tostring(TargetResources[0].userPrincip
      Role      = tostring(TargetResources[0].modifiedPro
| project TimeGenerated, InitiatedBy, Target, Role
```

Time chart of activity

```
SecurityEvent
| where TimeGenerated > ago(7d)
| summarize count() by bin(TimeGenerated, 1h), Computer
| render timechart
```

`bin()` + `render timechart` is the standard workbook pattern.

Match logs against threat intelligence (a `join`)

```
let indicators = ThreatIntelligenceIndicator
    | where ExpirationDateTime > now() and Active == true
    | where isnotempty(NetworkIP)
    | project NetworkIP, Description;
CommonSecurityLog
| where TimeGenerated > ago(1d)
| join kind=inner indicators on $left.DestinationIP == $right.DestinationIP
```

`join kind=inner` keeps only matches. Other kinds: `leftouter`, `rightouter`, `fullouter`, `leftanti` (**rows in the left table with NO match** — useful for "which servers are NOT reporting").

Machines that stopped sending data

```
Heartbeat
| summarize LastSeen = max(TimeGenerated) by Computer
| where LastSeen < ago(1h)
```

Which table is costing the most

```
Usage
| where TimeGenerated > ago(30d)
| summarize TotalGB = sum(Quantity) / 1000 by DataType
```

```
| sort by TotalGB desc
```

let for a reusable list (watchlist-style)

```
let VIPs = dynamic(["ceo@contoso.com", "cfo@contoso.com"]  
SigninLogs  
| where UserPrincipalName in (VIPs)  
| where ResultType != 0
```

The real Sentinel equivalent uses a **watchlist**:

```
_GetWatchlist('VIPUsers').
```

5. DCR ingestion-time transformations

Worth recognising, because the outline names DCRs. A transformation is a **KQL fragment applied at ingestion** to drop noise or reshape data before it is billed and stored:

```
source  
| where EventID in (4624, 4625, 4672, 4688)  
| project TimeGenerated, Computer, Account, EventID, IpAd
```

- The pipeline always starts from **source**.
 - Use it to **cut ingestion cost**, remove sensitive fields before storage, or normalise a custom log.
 - Supported operators are a subset — no `join` across tables, no cross-resource queries.
-

6. What NOT to spend time on

Skip: `mv-apply`, `series_decompose_anomalies`, `make-series`,

custom functions, `externaldata`, ASIM parsers in depth, and notebook/MSTICPy material. They are real Sentinel skills but not AZ-500 content.

If you can do these five things you are done:

1. Point at a table name and say what data it holds.
2. Read a 6-line query and state its intent in one sentence.
3. Say what `summarize ... by, project, extend, and bin()` each do.
4. Know that `where on TimeGenerated` goes **first**.
5. Know `AzureActivity = control plane`, `SecurityEvent = Windows guest`, `SigninLogs/AuditLogs = Entra`.